SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - Database Design and Connectivity

Course Code:	DSA0501	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	**AT1 – Database Design and Connectivity**
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	**DENISHA SHREE R**	**Reg. No.:**	**192524016**
Section / Batch:	**AIDS/2025**	**Date of Submission:**	**29-07-2026**
Faculty In charge:	K Veena devi	**Project Mode:**	Individual Submission

Code of Conduct

I __DENISHA SHREE R(192524016)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: __DENISHA SHREE R___

Requirement Analysis (4 Marks)	ER Diagram Design	Table Design & Normalizatio	Database Connectivity using Python	CRUD Operations (4 Marks)	Total (20 Marks)

	(4 Marks)	n (4 Marks)	(4 Marks)		

CO2 – AT1 – Database Design and Connectivity

Sample Questions

1. Student Database Design and Connectivity (10 Marks)

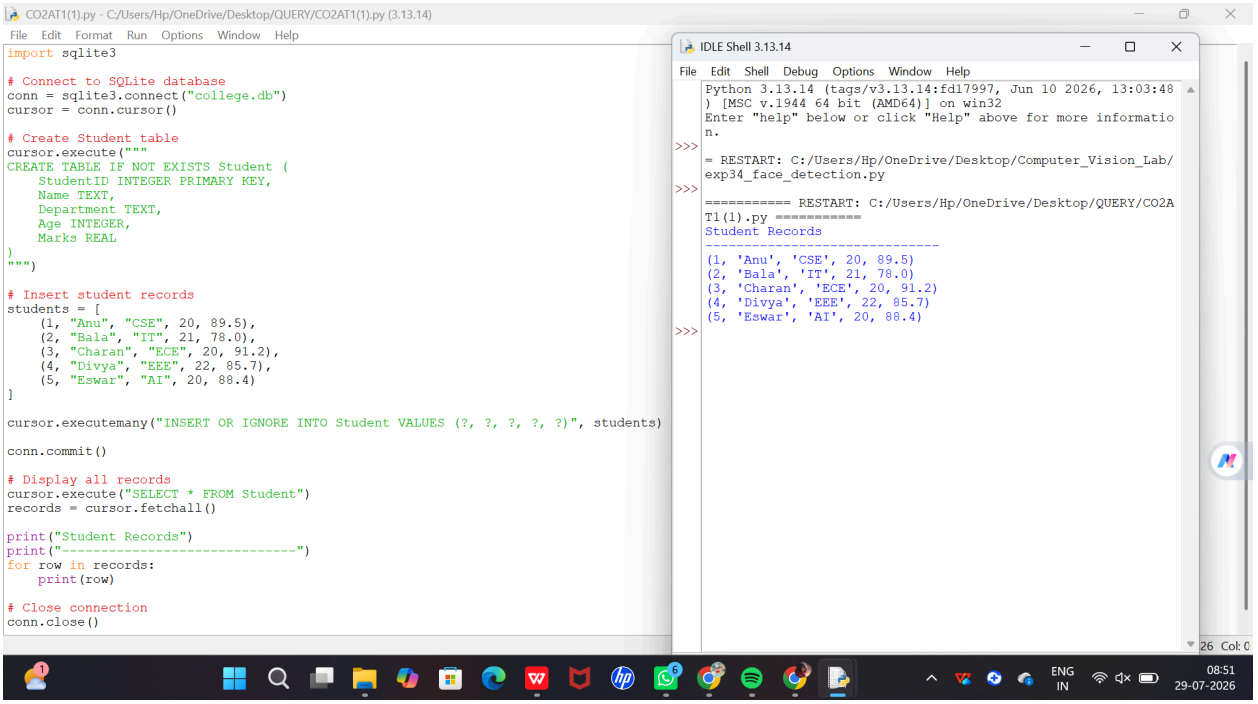
A college wants to automate the management of student records. Design an SQLite database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the SQLite database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

# Create Student table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Student (

   StudentID INTEGER PRIMARY KEY,

   Name TEXT,

   Department TEXT,

   Age INTEGER,

   Marks REAL

)

""")

# Insert student records

students = [

   (1, "Anu", "CSE", 20, 89.5),
```

```
    (2, "Bala", "IT", 21, 78.0),
    (3, "Charan", "ECE", 20, 91.2),
    (4, "Divya", "EEE", 22, 85.7),
    (5, "Eswar", "AI", 20, 88.4)
]
cursor.executemany("INSERT OR IGNORE INTO Student VALUES (?, ?, ?, ?, ?)", students)
conn.commit()
# Display all records
cursor.execute("SELECT * FROM Student")
records = cursor.fetchall()
print("Student Records")
print("------------------------------")
for row in records:
    print(row)
# Close connection
conn.close()
```

OUTPUT:



2. Library Management Database (10 Marks)

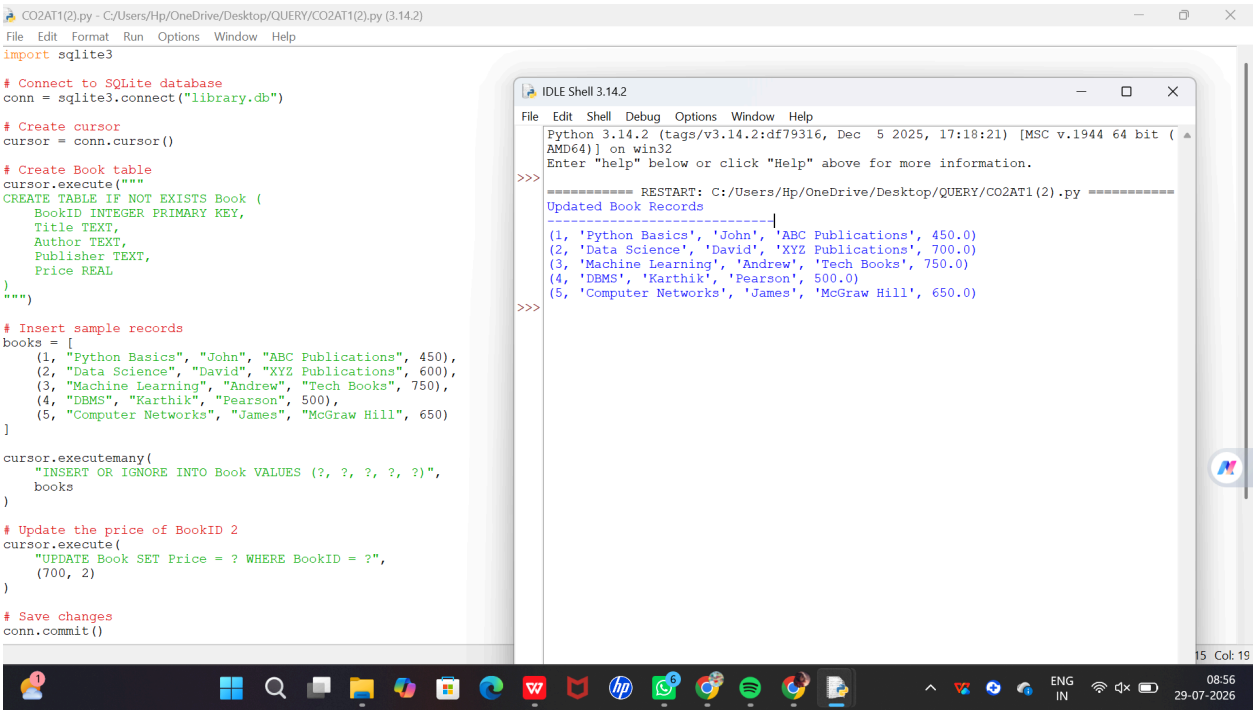
A library plans to maintain information about its books using an SQLite database. Design a **Book** table containing suitable attributes such as Book ID, Title, Author, Publisher, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("library.db")

# Create cursor

cursor = conn.cursor()

# Create Book table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Book (

    BookID INTEGER PRIMARY KEY,

    Title TEXT,

    Author TEXT,

    Publisher TEXT,

    Price REAL

)

""")

# Insert sample records

books = [

    (1, "Python Basics", "John", "ABC Publications", 450),

    (2, "Data Science", "David", "XYZ Publications", 600),

    (3, "Machine Learning", "Andrew", "Tech Books", 750),

    (4, "DBMS", "Karthik", "Pearson", 500),
```

```
    (5, "Computer Networks", "James", "McGraw Hill", 650)
]
cursor.executemany(
    "INSERT OR IGNORE INTO Book VALUES (?, ?, ?, ?, ?)",
    books
)
# Update the price of BookID 2
cursor.execute(
    "UPDATE Book SET Price = ? WHERE BookID = ?",
    (700, 2)
)
# Save changes
conn.commit()
# Display updated records
cursor.execute("SELECT * FROM Book")
records = cursor.fetchall()
print("Updated Book Records")
print("-----------------------------")
for row in records:
    print(row)
# Close database
conn.close()
```

OUTPUT:



3. Employee and Department Database Design (10 Marks)

A company wants to store employee information and department details in an SQLite database. Design two related tables, **Department** and **Employee**, by identifying appropriate primary and foreign keys. Write a Python program to connect to the database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using an SQL JOIN query.

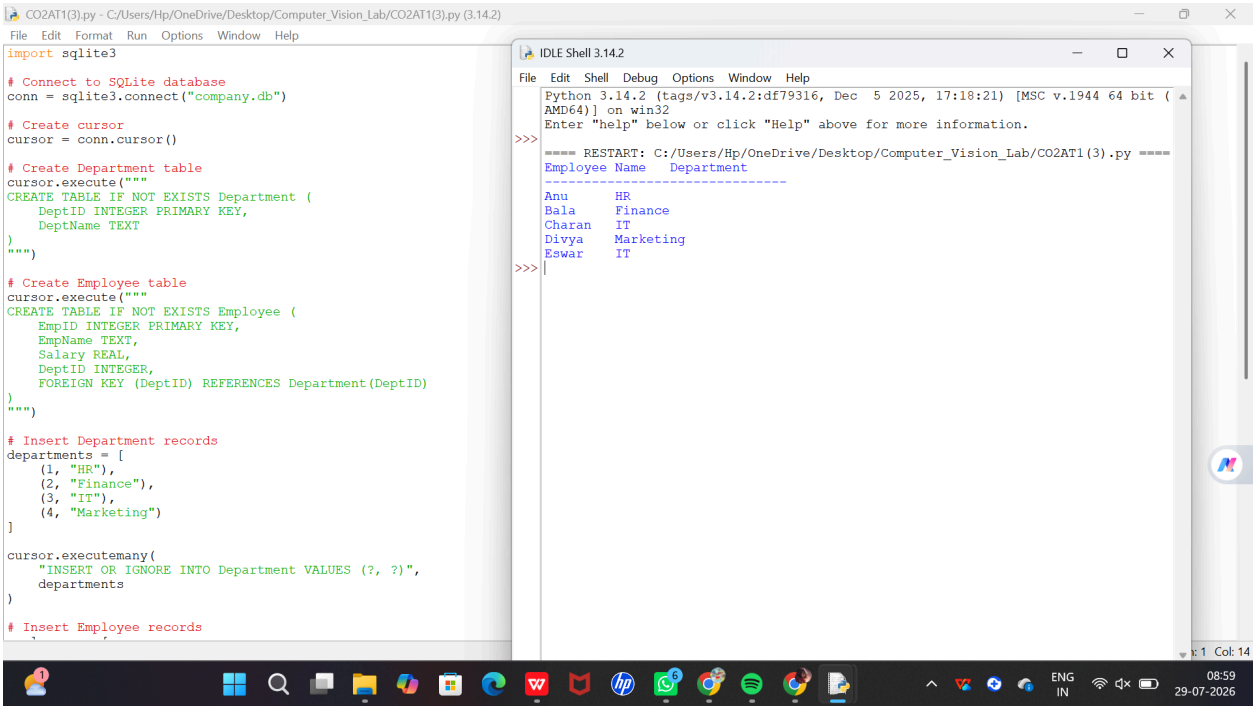
CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("company.db")

# Create cursor

cursor = conn.cursor()

# Create Department table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Department (

    DeptID INTEGER PRIMARY KEY,

    DeptName TEXT
```

```
)
""")
# Create Employee table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Employee (
    EmpID INTEGER PRIMARY KEY,
    EmpName TEXT,
    Salary REAL,
    DeptID INTEGER,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
)
""")
# Insert Department records
departments = [
    (1, "HR"),
    (2, "Finance"),
    (3, "IT"),
    (4, "Marketing")
]
cursor.executemany(
    "INSERT OR IGNORE INTO Department VALUES (?, ?)",
    departments
)
# Insert Employee records
employees = [
    (101, "Anu", 30000, 1),
    (102, "Bala", 45000, 2),
```

```
    (103, "Charan", 50000, 3),
    (104, "Divya", 40000, 4),
    (105, "Eswar", 55000, 3)
]
cursor.executemany(
    "INSERT OR IGNORE INTO Employee VALUES (?, ?, ?, ?)",
    employees
)
# Save changes
conn.commit()
# JOIN query
cursor.execute("""
SELECT Employee.EmpName, Department.DeptName
FROM Employee
INNER JOIN Department
ON Employee.DeptID = Department.DeptID
""")
records = cursor.fetchall()
print("Employee Name\tDepartment")
print("-------------------------------")
for row in records:
    print(row[0], "\t", row[1])
# Close database
conn.close()
```

OUTPUT:



4. Hospital Patient Database (10 Marks)

A hospital requires a database to manage patient information. Design an SQLite database with a **Patient** table containing suitable fields and constraints. Write a Python program to connect to the database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

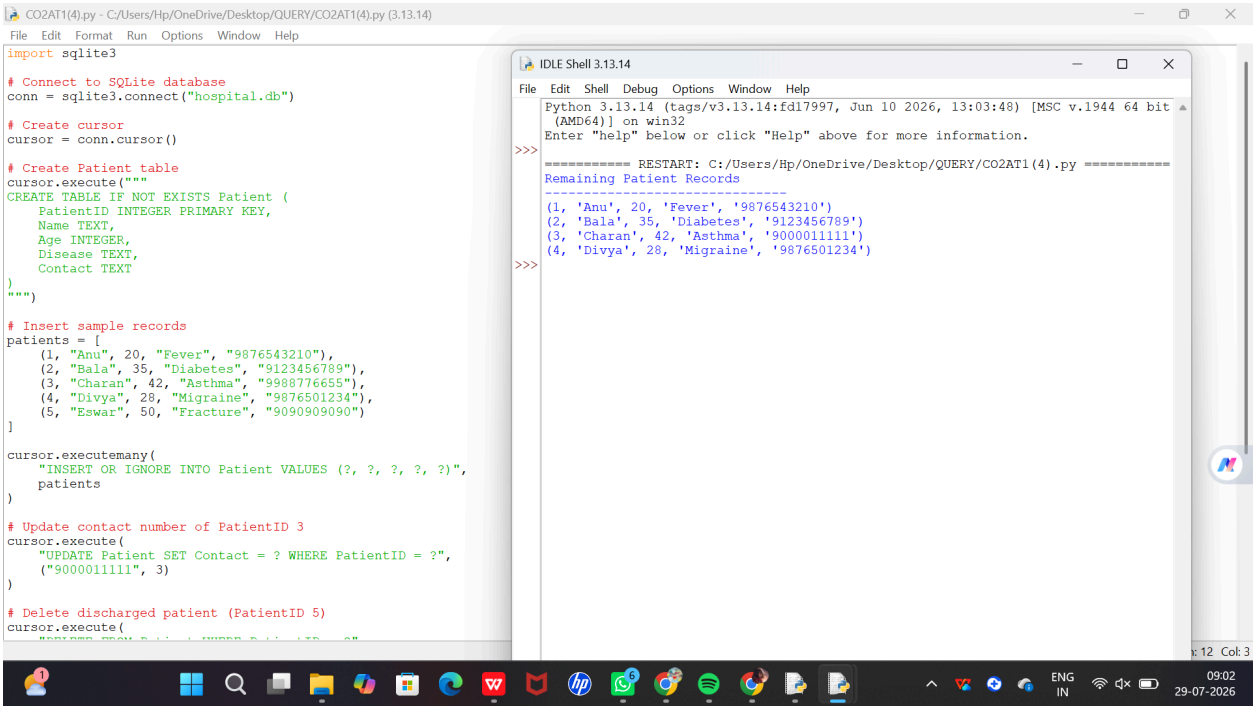
CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("hospital.db")

# Create cursor

cursor = conn.cursor()

# Create Patient table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Patient (

    PatientID INTEGER PRIMARY KEY,

    Name TEXT,
```

```
    Age INTEGER,
    Disease TEXT,
    Contact TEXT
)
""")
# Insert sample records
patients = [
    (1, "Anu", 20, "Fever", "9876543210"),
    (2, "Bala", 35, "Diabetes", "9123456789"),
    (3, "Charan", 42, "Asthma", "9988776655"),
    (4, "Divya", 28, "Migraine", "9876501234"),
    (5, "Eswar", 50, "Fracture", "9090909090")
]
cursor.executemany(
    "INSERT OR IGNORE INTO Patient VALUES (?, ?, ?, ?, ?)",
    patients
)
# Update contact number of PatientID 3
cursor.execute(
    "UPDATE Patient SET Contact = ? WHERE PatientID = ?",
    ("9000011111", 3)
)
# Delete discharged patient (PatientID 5)
cursor.execute(
    "DELETE FROM Patient WHERE PatientID = ?",
    (5,)
)
```

```
# Save changes

conn.commit()

# Display remaining records

cursor.execute("SELECT * FROM Patient")

records = cursor.fetchall()

print("Remaining Patient Records")

print("------------------------------")

for row in records:

    print(row)

# Close database

conn.close()
```

OUTPUT:



5. Online Course Registration System (10 Marks)

An online learning platform needs to maintain information about students and the courses they have registered for. Design two related SQLite tables, **Student** and **Course_Registration**, using appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample records, and retrieve the names of students along

with the courses they have registered for using an SQL JOIN operation.

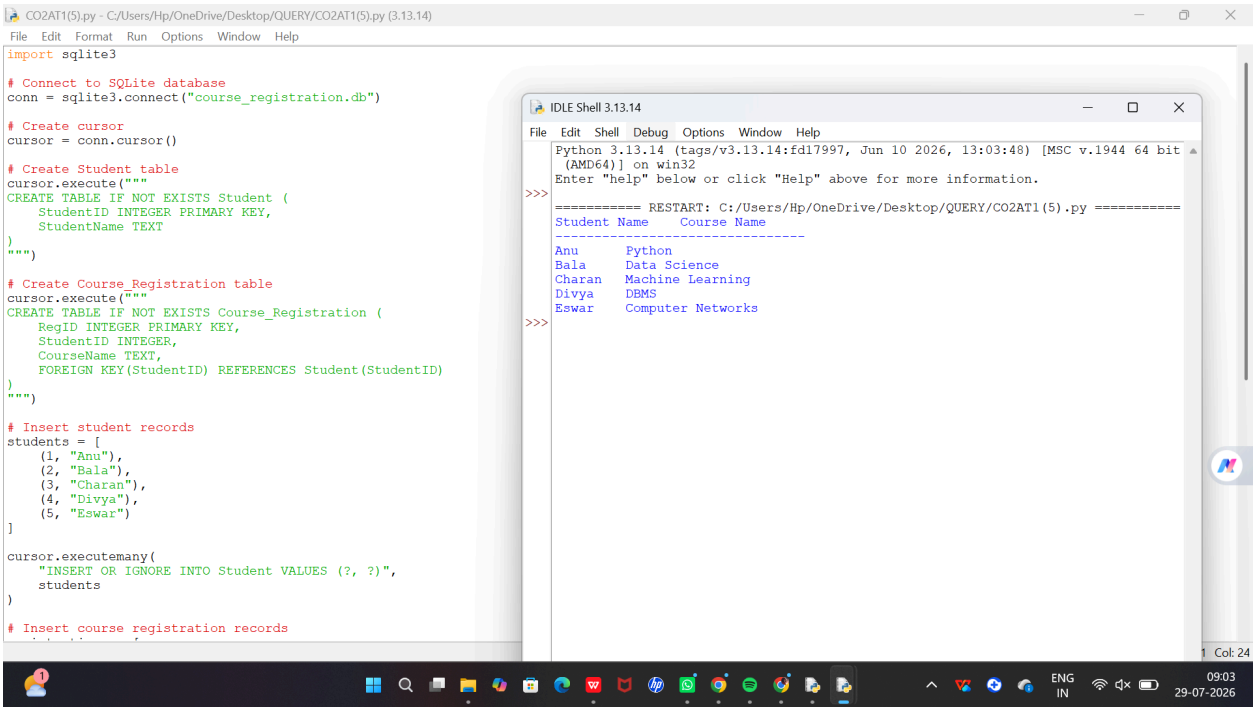
CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("course_registration.db")

# Create cursor

cursor = conn.cursor()

# Create Student table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Student (

   StudentID INTEGER PRIMARY KEY,

   StudentName TEXT

)

""")

# Create Course_Registration table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Course_Registration (

   RegID INTEGER PRIMARY KEY,

   StudentID INTEGER,

   CourseName TEXT,

   FOREIGN KEY(StudentID) REFERENCES Student(StudentID)

)

""")

# Insert student records

students = [

   (1, "Anu"),

   (2, "Bala"),
```

```
    (3, "Charan"),
    (4, "Divya"),
    (5, "Eswar")
]
cursor.executemany(
    "INSERT OR IGNORE INTO Student VALUES (?, ?)",
    students
)
# Insert course registration records
registrations = [
    (101, 1, "Python"),
    (102, 2, "Data Science"),
    (103, 3, "Machine Learning"),
    (104, 4, "DBMS"),
    (105, 5, "Computer Networks")
]
cursor.executemany(
    "INSERT OR IGNORE INTO Course_Registration VALUES (?, ?, ?)",
    registrations
)
# Save changes
conn.commit()
# JOIN query
cursor.execute("""
SELECT Student.StudentName, Course_Registration.CourseName
FROM Student
INNER JOIN Course_Registration
```

```
ON Student.StudentID = Course_Registration.StudentID

""")

records = cursor.fetchall()

print("Student Name\tCourse Name")

print("--------------------------------")

for row in records:

    print(row[0], "\t", row[1])

# Close database

conn.close()
```

OUTPUT:



6. Banking Account Management System (10 Marks)

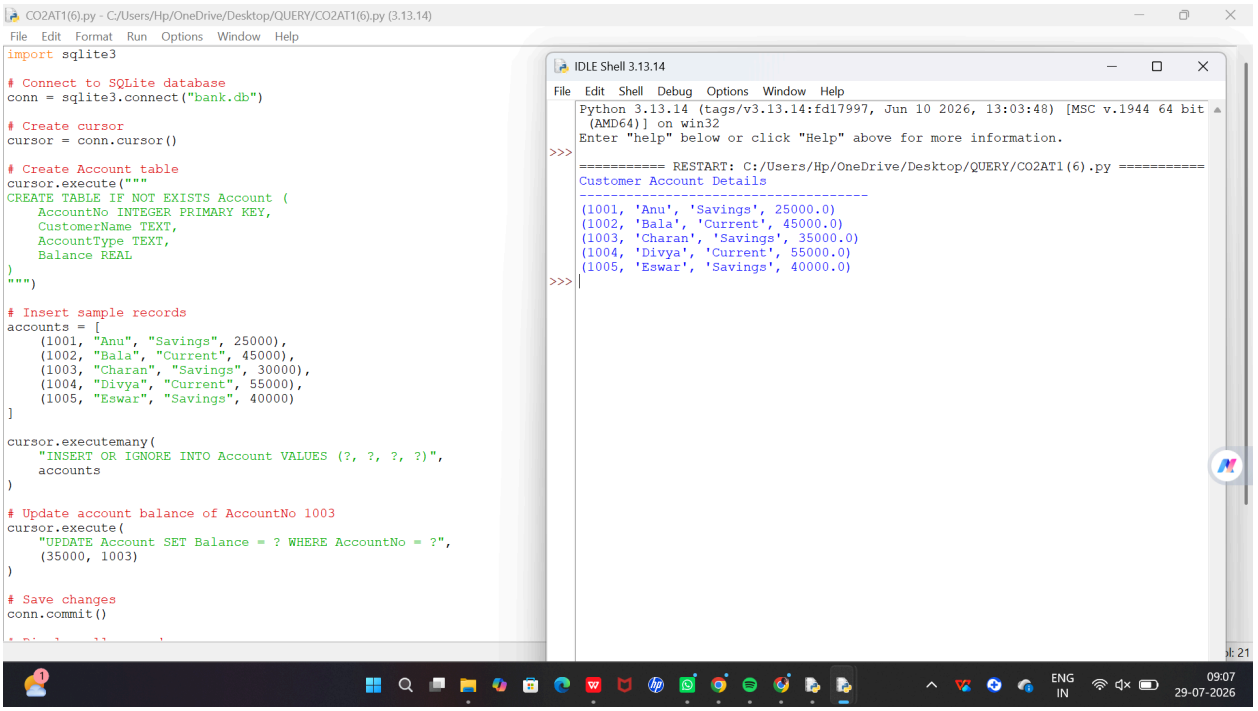
A bank wants to maintain customer account information using an SQLite database. Design an appropriate database containing an **Account** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert customer account details, update the account balance of a specified customer, and display all customer records.

CODE:

```
import sqlite3
# Connect to SQLite database
conn = sqlite3.connect("bank.db")
```

```
# Create cursor
cursor = conn.cursor()
# Create Account table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Account (
   AccountNo INTEGER PRIMARY KEY,
   CustomerName TEXT,
   AccountType TEXT,
   Balance REAL
)
""")
# Insert sample records
accounts = [
   (1001, "Anu", "Savings", 25000),
   (1002, "Bala", "Current", 45000),
   (1003, "Charan", "Savings", 30000),
   (1004, "Divya", "Current", 55000),
   (1005, "Eswar", "Savings", 40000)
]
cursor.executemany(
   "INSERT OR IGNORE INTO Account VALUES (?, ?, ?, ?)",
   accounts
)
# Update account balance of AccountNo 1003
cursor.execute(
   "UPDATE Account SET Balance = ? WHERE AccountNo = ?",
   (35000, 1003)
)
# Save changes
conn.commit()
# Display all records
cursor.execute("SELECT * FROM Account")
records = cursor.fetchall()
print("Customer Account Details")
print("-------------------------------------")
for row in records:
   print(row)

# Close database
conn.close()
```

OUTPUT:



7. Inventory Management System (10 Marks)

A retail store needs to maintain product inventory using SQLite. Design a **Product** table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to create the database, insert sample products, identify products with quantity less than 10, update their stock quantity, and display the updated inventory.

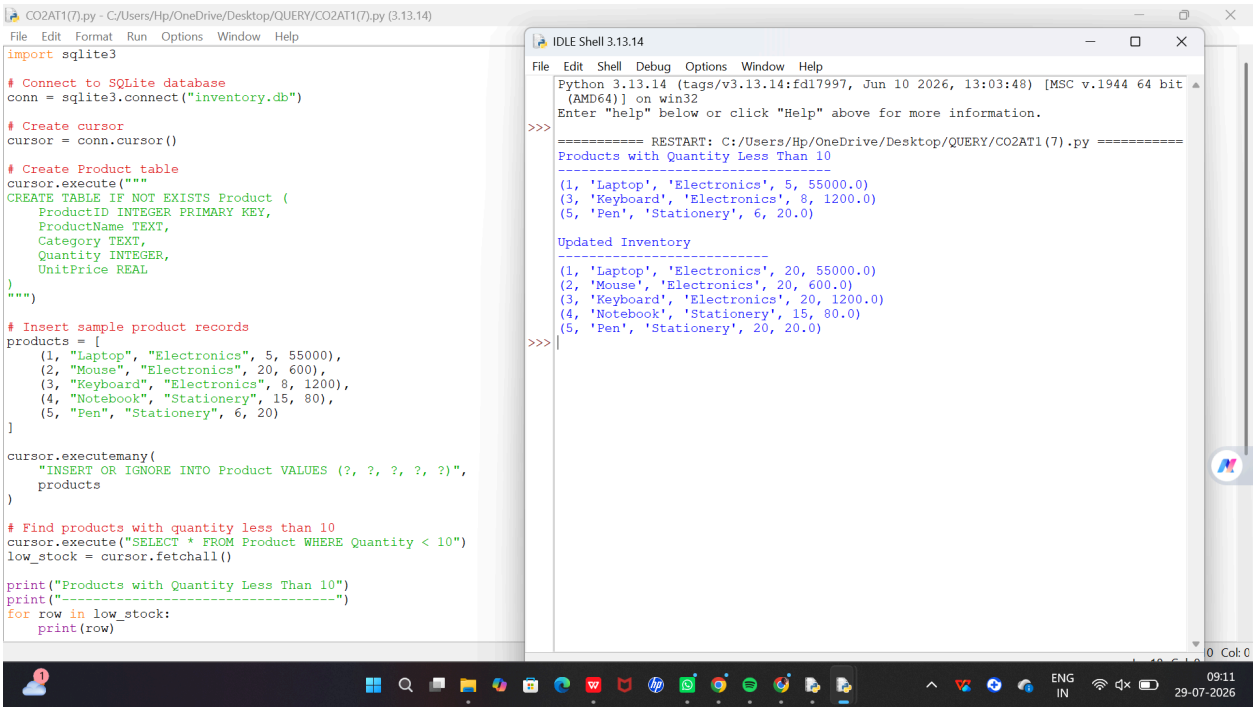
CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("inventory.db")

# Create cursor

cursor = conn.cursor()

# Create Product table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Product (

    ProductID INTEGER PRIMARY KEY,

    ProductName TEXT,

    Category TEXT,
```

```
    Quantity INTEGER,
    UnitPrice REAL
)
""")
# Insert sample product records
products = [
    (1, "Laptop", "Electronics", 5, 55000),
    (2, "Mouse", "Electronics", 20, 600),
    (3, "Keyboard", "Electronics", 8, 1200),
    (4, "Notebook", "Stationery", 15, 80),
    (5, "Pen", "Stationery", 6, 20)
]
cursor.executemany(
    "INSERT OR IGNORE INTO Product VALUES (?, ?, ?, ?, ?)",
    products
)
# Find products with quantity less than 10
cursor.execute("SELECT * FROM Product WHERE Quantity < 10")
low_stock = cursor.fetchall()
print("Products with Quantity Less Than 10")
print("----------------------------------")
for row in low_stock:
    print(row)
# Update stock quantity to 20
cursor.execute("UPDATE Product SET Quantity = 20 WHERE Quantity < 10")
# Save changes
conn.commit()
```

```
# Display updated inventory

cursor.execute("SELECT * FROM Product")

records = cursor.fetchall()

print("\nUpdated Inventory")

print("--------------------------")

for row in records:

    print(row)

# Close database

conn.close()
```

OUTPUT:



8. Course and Faculty Management System (10 Marks)

A university wants to maintain details of faculty members and the courses they teach. Design two related tables, **Faculty** and **Course**, using appropriate primary and foreign keys. Write a Python program to create the database, insert sample records, and display each faculty member along with the courses assigned using an SQL JOIN query.

CODE:

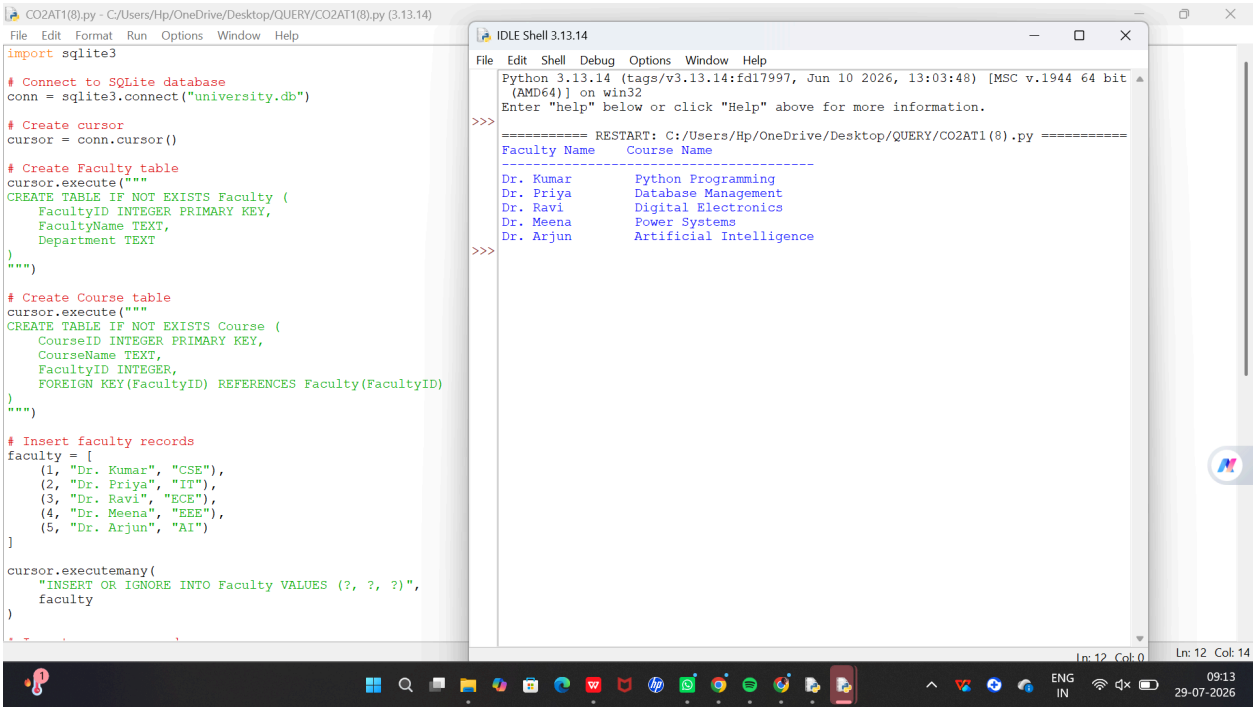
```
import sqlite3
```

```
# Connect to SQLite database
conn = sqlite3.connect("university.db")
# Create cursor
cursor = conn.cursor()
# Create Faculty table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Faculty (
    FacultyID INTEGER PRIMARY KEY,
    FacultyName TEXT,
    Department TEXT
)
""")
# Create Course table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Course (
    CourseID INTEGER PRIMARY KEY,
    CourseName TEXT,
    FacultyID INTEGER,
    FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID)
)
""")
# Insert faculty records
faculty = [
    (1, "Dr. Kumar", "CSE"),
    (2, "Dr. Priya", "IT"),
    (3, "Dr. Ravi", "ECE"),
    (4, "Dr. Meena", "EEE"),
```

```
    (5, "Dr. Arjun", "AI")
]
cursor.executemany(
    "INSERT OR IGNORE INTO Faculty VALUES (?, ?, ?)",
    faculty
)
# Insert course records
courses = [
    (101, "Python Programming", 1),
    (102, "Database Management", 2),
    (103, "Digital Electronics", 3),
    (104, "Power Systems", 4),
    (105, "Artificial Intelligence", 5)
]
cursor.executemany(
    "INSERT OR IGNORE INTO Course VALUES (?, ?, ?)",
    courses
)
# Save changes
conn.commit()
# JOIN query
cursor.execute("""
SELECT Faculty.FacultyName, Course.CourseName
FROM Faculty
INNER JOIN Course
ON Faculty.FacultyID = Course.FacultyID
""")
```

```
records = cursor.fetchall()

print("Faculty Name\tCourse Name")

print("----------------------------------------")

for row in records:

    print(row[0], "\t", row[1])

# Close database

conn.close()
```

OUTPUT:



9. Vehicle Registration Database (10 Marks)

A transport department plans to maintain vehicle registration details in an SQLite database. Design a **Vehicle** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert vehicle records, search for a vehicle using its registration number, and display the retrieved information.

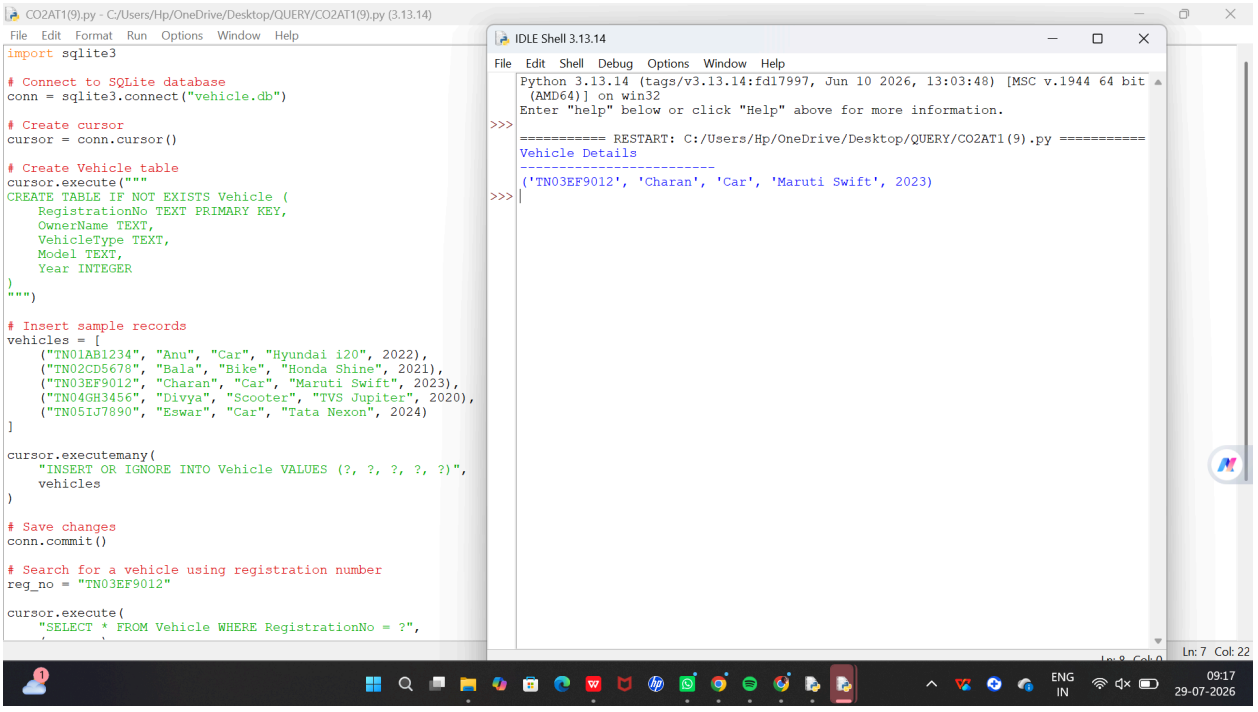
CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("vehicle.db")
```

```
# Create cursor
cursor = conn.cursor()
# Create Vehicle table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Vehicle (
    RegistrationNo TEXT PRIMARY KEY,
    OwnerName TEXT,
    VehicleType TEXT,
    Model TEXT,
    Year INTEGER
)
""")
# Insert sample records
vehicles = [
    ("TN01AB1234", "Anu", "Car", "Hyundai i20", 2022),
    ("TN02CD5678", "Bala", "Bike", "Honda Shine", 2021),
    ("TN03EF9012", "Charan", "Car", "Maruti Swift", 2023),
    ("TN04GH3456", "Divya", "Scooter", "TVS Jupiter", 2020),
    ("TN05IJ7890", "Eswar", "Car", "Tata Nexon", 2024)
]
cursor.executemany(
    "INSERT OR IGNORE INTO Vehicle VALUES (?, ?, ?, ?, ?)",
    vehicles
)
# Save changes
conn.commit()
# Search for a vehicle using registration number
```

```
reg_no = "TN03EF9012"

cursor.execute(

    "SELECT * FROM Vehicle WHERE RegistrationNo = ?",

    (reg_no,)

)

record = cursor.fetchone()

print("Vehicle Details")

print("------------------------")

if record:

    print(record)

else:

    print("Vehicle not found")

# Close database

conn.close()
```

OUTPUT:



10. Hospital Appointment Management System (10 Marks)

A hospital maintains separate tables for **Doctors** and **Appointments**. Design both tables using appropriate primary and foreign keys. Write a Python program to create the tables, insert sample records, and display the doctor name along with the appointment details using a JOIN query.

CODE:

```
import sqlite3

# Connect to SQLite database

conn = sqlite3.connect("hospital_appointment.db")

# Create cursor

cursor = conn.cursor()

# Create Doctor table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Doctor (

    DoctorID INTEGER PRIMARY KEY,

    DoctorName TEXT,

    Specialization TEXT

)

""")

# Create Appointment table

cursor.execute("""

CREATE TABLE IF NOT EXISTS Appointment (

    AppointmentID INTEGER PRIMARY KEY,

    PatientName TEXT,

    AppointmentDate TEXT,

    DoctorID INTEGER,

    FOREIGN KEY(DoctorID) REFERENCES Doctor(DoctorID)
```

```
)
""")
# Insert Doctor records
doctors = [
    (1, "Dr. Kumar", "Cardiologist"),
    (2, "Dr. Priya", "Dermatologist"),
    (3, "Dr. Ravi", "Orthopedic"),
    (4, "Dr. Meena", "Neurologist"),
    (5, "Dr. Arjun", "Pediatrician")
]
cursor.executemany(
    "INSERT OR IGNORE INTO Doctor VALUES (?, ?, ?)",
    doctors
)
# Insert Appointment records
appointments = [
    (101, "Anu", "2026-08-01", 1),
    (102, "Bala", "2026-08-02", 2),
    (103, "Charan", "2026-08-03", 3),
    (104, "Divya", "2026-08-04", 4),
    (105, "Eswar", "2026-08-05", 5)
]
cursor.executemany(
    "INSERT OR IGNORE INTO Appointment VALUES (?, ?, ?, ?)",
    appointments
)
# Save changes
```

```
conn.commit()

# JOIN query

cursor.execute("""

SELECT Doctor.DoctorName,

      Appointment.PatientName,

      Appointment.AppointmentDate

FROM Doctor

INNER JOIN Appointment

ON Doctor.DoctorID = Appointment.DoctorID

""")

records = cursor.fetchall()

print("Doctor Name\tPatient Name\tAppointment Date")

print("-------------------------------------------------------")

for row in records:

   print(row[0], "\t", row[1], "\t", row[2])

# Close database

conn.close()
```

OUTPUT:

CO2AT1(10).py - C:/Users/Hp/OneDrive/Desktop/QUERY/CO2AT1(10).py (3.13.14)

File Edit Format Run Options Window Help

```
import sqlite3

# Connect to SQLite database
conn = sqlite3.connect("hospital_appointment.db")

# Create cursor
cursor = conn.cursor()

# Create Doctor table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Doctor (
    DoctorID INTEGER PRIMARY KEY,
    DoctorName TEXT,
    Specialization TEXT
)
""")

# Create Appointment table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Appointment (
    AppointmentID INTEGER PRIMARY KEY,
    PatientName TEXT,
    AppointmentDate TEXT,
    DoctorID INTEGER,
    FOREIGN KEY(DoctorID) REFERENCES Doctor(DoctorID)
)
""")

# Insert Doctor records
doctors = [
    (1, "Dr. Kumar", "Cardiologist"),
    (2, "Dr. Priya", "Dermatologist"),
    (3, "Dr. Ravi", "Orthopedic"),
    (4, "Dr. Meena", "Neurologist"),
    (5, "Dr. Arjun", "Pediatrician")
]

cursor.executemany(
    "INSERT OR IGNORE INTO Doctor VALUES (?, ?, ?)",
    doctors
)
```

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13
 (AMD64)] on win32
Enter "help" below or click "Help" above for more info
>>>

========== RESTART: C:/Users/Hp/OneDrive/Desktop/QUER
Doctor Name      Patient Name    Appointment Date
------------------------------------------------------------
Dr. Kumar        Anu     2026-08-01
Dr. Priya        Bala    2026-08-02
Dr. Ravi         Charan          2026-08-03
Dr. Meena        Divya   2026-08-04
Dr. Arjun        Eswar   2026-08-05
>>>
```